

**ESCOLA
SUPERIOR
DE TECNOLOGIA
E GESTÃO**

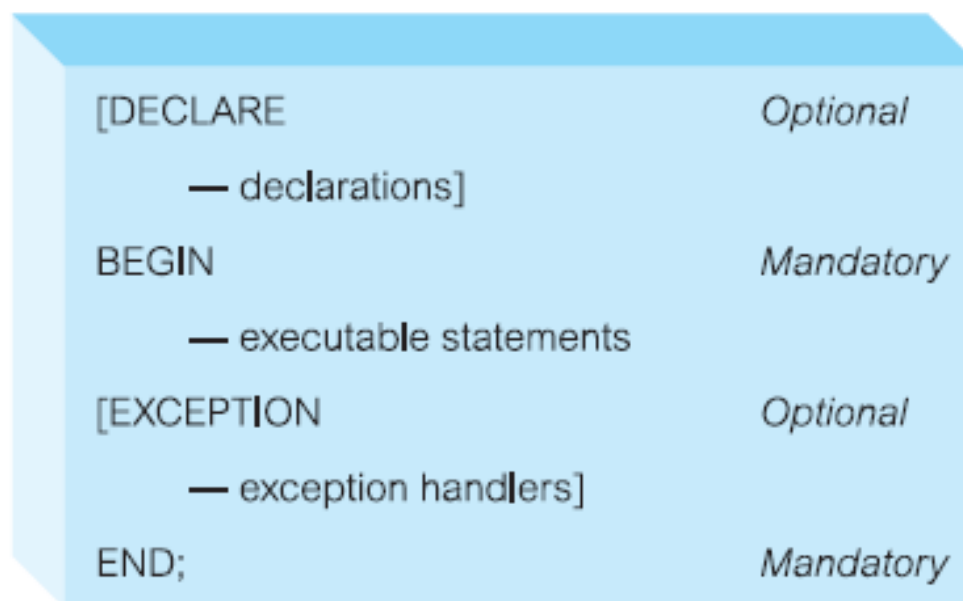
P.PORTO

SQL AVANÇADO

SQL AVANÇADO

- A Linguagem SQL também tem uma componente programática.
 - Permite complementar e estender as funcionalidades da linguagem SQL
 - **PL/SQL** – Procedural Language/SQL (Oracle)
 - **T-SQL** – Transact-SQL (Microsoft SQL Server)
- Ambas as extensões assentam na estrutura típica de uma linguagem imperativa.

Estrutura PL/SQL



Declarações

- Variáveis e Constantes têm que ser declaradas antes de serem utilizadas
- É possível declarar uma variável como:
 - NOT NULL
 - %TYPE (de um atributo existente numa tabela)
 - %ROWTYPE (de um registo de uma tabela)

Atribuições

- Pode-se atribuir valores a variáveis através:
 - do operador (:)
 - de uma instrução SQL SELECT ou FETCH

Instruções de Controlo de Fluxo

- Condicionais:
 - IF
 - CASE
- Repetitivas
 - LOOP
 - WHILE e REPEAT
 - FOR

Exceções em PL/SQL

- **EXCEPTION**
 - é um identificador em PL/SQL
 - capturada na execução de um bloco de instruções
 - termina a definição do bloco de ações
- O programador define no bloco da EXCEPTION quais as ações a despoletar no caso de haver uma exceção.

Exemplo de Exceções em PL/SQL

```
DECLARE
    vpCount    NUMBER;
    vStaffNo PropertyForRent.staffNo%TYPE := 'SG14';
    -- define an exception for the enterprise constraint that prevents a member of staff
    -- managing more than 100 properties
    e_too_many_properties EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_too_many_properties, -20000);
BEGIN
    SELECT COUNT(*) INTO vpCount
    FROM PropertyForRent
    WHERE staffNo = vStaffNo;
    IF vpCount = 100
    -- raise an exception for the general constraint
        RAISE e_too_many_properties;
    END IF;
    UPDATE PropertyForRent SET staffNo = vStaffNo WHERE propertyNo = 'PG4';
EXCEPTION
    -- handle the exception for the general constraint
    WHEN e_too_many_properties THEN
        dbms_output.put_line('Member of staff ' || staffNo || 'already managing 100 properties');
END;
```

Cursors em PL/SQL

- CURSOR
 - Permite que, ao resultado de uma query, se aceda a um registo de cada vez .
 - Tem que ser declarado e aberto antes de usado
 - Tem que ser fechado depois de já não ser necessário para libertar recursos

Exemplo de Cursores em PL/SQL

```
DECLARE
    vPropertyNo    PropertyForRent.propertyNo%TYPE;
    vStreet        PropertyForRent.street%TYPE;
    vCity          PropertyForRent.city%TYPE;
    vPostcode      PropertyForRent.postcode%TYPE;
    CURSOR propertyCursor IS
        SELECT propertyNo, street, city, postcode
        FROM PropertyForRent
        WHERE staffNo = 'SG14'
        ORDER by propertyNo;

BEGIN
    -- Open the cursor to start of selection, then loop to fetch each row of the result table
    OPEN propertyCursor;
    LOOP

    -- Fetch next row of the result table
        FETCH propertyCursor
            INTO vPropertyNo, vStreet, vCity, vPostcode;
        EXIT WHEN propertyCursor%NOTFOUND;

    -- Display data
        dbms_output.put_line('Property number: ' || vPropertyNo);
        dbms_output.put_line('Street:      ' || vStreet);
        dbms_output.put_line('City:      ' || vCity);
        IF postcode IS NOT NULL THEN
            dbms_output.put_line('Post Code:      ' || vPostcode);
        ELSE
            dbms_output.put_line('Post Code:      NULL');
        END IF;
    END LOOP;
    IF propertyCursor%ISOPEN THEN CLOSE propertyCursor END IF;

    -- Error condition - print out error
EXCEPTION
    WHEN OTHERS THEN
        dbms_output.put_line('Error detected');
        IF propertyCursor%ISOPEN THEN CLOSE propertyCursor; END IF;
END;
```

Subprogramas

- Subprogramas
 - Blocos em PL/SQL que aceitam parâmetros e podem ser chamados
 - Tipos
 - Stored Procedures
 - Functions

Triggers

- **TRIGGER**
 - Define as ações a executar quando um evento ocorre numa tabela
 - Tipos de Triggers
 - Vantagens e Desvantagens da sua utilização

Sintaxe do TRIGGER

```
CREATE TRIGGER TriggerName  
  BEFORE | AFTER | INSTEAD OF  
  INSERT | DELETE | UPDATE [OF TriggerColumnList]  
  
  ON TableName  
  [REFERENCING {OLD | NEW} AS {OldName | NewName}]  
  [FOR EACH {ROW | STATEMENT}]  
  [WHEN Condition]  
  <trigger action>
```

Exemplo de um BEFORE TRIGGER

```
CREATE TRIGGER StaffNotHandlingTooMuch
BEFORE INSERT ON PropertyForRent
REFERENCING NEW AS newrow
FOR EACH ROW
DECLARE
    vpCount    NUMBER;
BEGIN
    SELECT COUNT(*) INTO vpCount
    FROM PropertyForRent
    WHERE staffNo = :newrow.staffNo;
    IF vpCount = 100
        raise_application_error(-20000, ('Member' || :newrow.staffNo || 'already managing 100 properties'));
    END IF;
END;
```

**ESCOLA
SUPERIOR
DE TECNOLOGIA
E GESTÃO**

P.PORTO

SQL AVANÇADO